Dataset Generation for Model Inductive Bias Identification

Anna Novokshonova

MIPT, PAMCS, Department of Intelligent Systems

Introduction

Inductive bias – a model's preference for certain data structures (e.g., CNNs favor locality and translation invariance, RNNs favor sequential order) – plays a fundamental role in a model's ability to generalize. However, understanding the correspondence between models and their inductive biases is crucial yet analytically intractable for most complex architectures.

Existing meta-learning approaches (Dorrell et al., 2023) generate only labels for samples from a fixed distribution, which assumes prior knowledge about the input data.

Our contribution: A generative meta-learning framework where a meta-learner is trained to produce synthetic easy-to-generalize datasets that reflect the target model's inductive bias.

Problem statement

Given a target model $\mathbf{f}_\theta : \mathbb{X} \to \mathbb{Y}$ with loss function $\ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y})$, the generator $\mathbf{g}(\mathbf{w})$ defines a parametric family of conditional distributions $p(\mathbf{x} \mid \mathbf{y}, \mathbf{w})$ over inputs $\mathbf{x}$ given labels $\mathbf{y}$.

Required to find data distribution function p^*, corresponding to parameters $\mathbf{w}^*$, the solution to the following optimization problem:

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} \; \mathbb{E}_{\mathcal{D}_{\text{syn}} \sim p(\mathbf{x}|\mathbf{y},\mathbf{w})} \left[\mathcal{L}\left(\mathbf{f}_{\hat{\theta}}, \mathcal{D}_{\text{syn}}\right) \right]$$

$$\text{s.t.} \quad \mathfrak{D}_{\text{syn}} = \mathfrak{D}_{\text{train}} \cup \mathfrak{D}_{\text{test}} \in \mathcal{U}$$

where $\hat{\theta} = \arg\min_{\theta \in \Theta} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{train}}} \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y})$ and $\mathcal{U}$ is the set of datasets consisting of c classes with n samples per class.

The meta-loss function $\mathcal{L}$ combines prediction error $\mathcal{L}_g$ and regularization $\mathcal{R}$:

$$\mathcal{L} = \mathcal{L}_g + \lambda \mathcal{R}$$

Prediction error $\mathcal{L}_g$ (MSE or cross-entropy):

$$\mathcal{L}_{\text{CE}} = -\frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{test}}} \sum_{i=1}^{m} y_i \log\left([\mathbf{f}_{\hat{\theta}}(\mathbf{x})]_i\right)$$

$$\mathcal{L}_{\text{MSE}} = \frac{1}{|\mathfrak{D}_{\text{test}}|} \sum_{(\mathbf{x},\mathbf{y}) \in \mathfrak{D}_{\text{test}}} \|\mathbf{y} - \mathbf{f}_{\hat{\theta}}(\mathbf{x})\|^2$$

Regularization $\mathcal{R}$:

$$\mathcal{R}_{\text{dist}} = -\frac{1}{C} \sum_{k=1}^{C} \frac{1}{n_k(n_k-1)} \sum_{\substack{i,j \in \mathcal{I}_k \\ i \neq j}} \|\mathbf{x}_i - \mathbf{x}_j\|_2^2$$

$$\mathcal{R}_{\text{svd}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{r_k} \sum_{i=1}^{r_k} \left(\max(\tau - \sigma_i^{(k)}, 0)\right)^2$$

$$\mathcal{R}_{l_1+\text{smooth}} = \sum_{i=1}^{T} |x_i| + \frac{1}{T-1} \sum_{i=1}^{T-1} (x_{i+1} - x_i)^2$$

RNNs

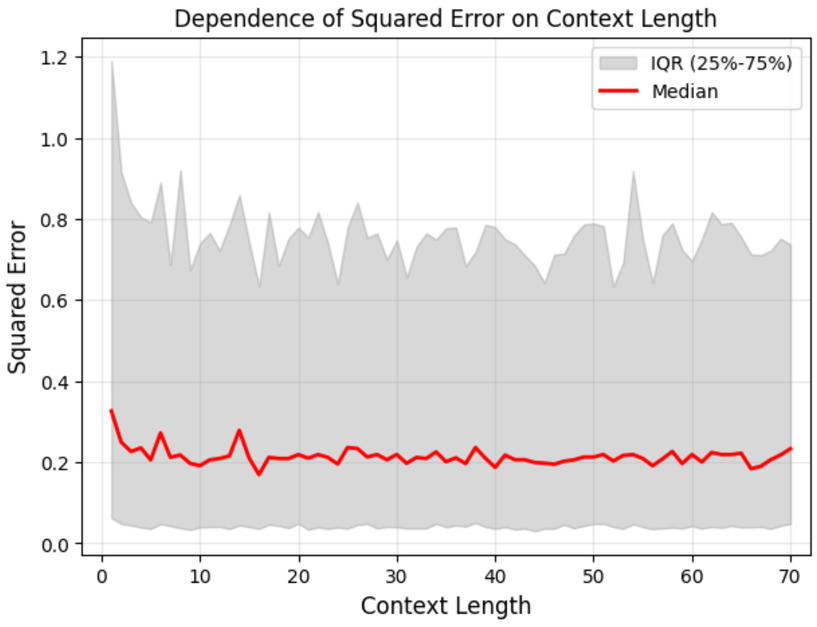

Figure: Dependence of squared error on context length.

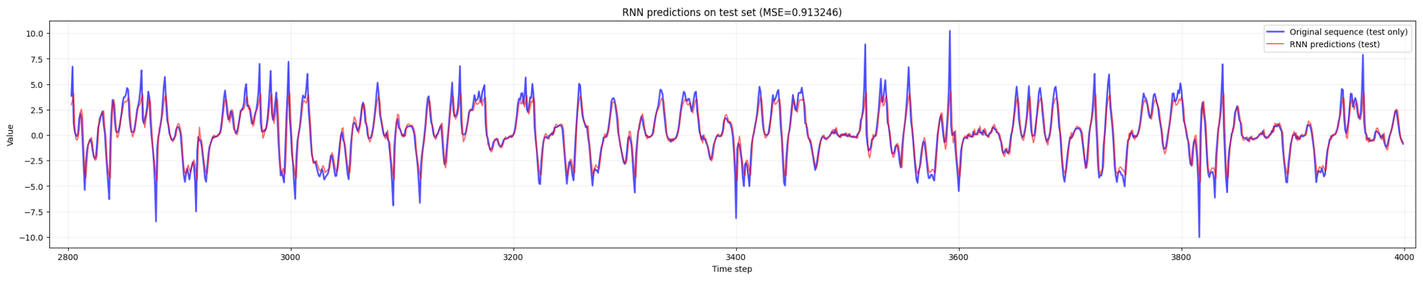

Figure: RNN prediction on test set.

RNN Architecture: Single RNN layer (hidden size 16) followed by a linear layer for next-step prediction.

Training	Test	MSE
Original	Original	$\mathbf{1.037 \pm 0.045}$
Original	Shuffled	$\mathbf{7.686 \pm 0.299}$
Shuffled	Shuffled	$\mathbf{7.136 \pm 0.008}$

Table: MSE comparison on original and shuffled data.

Sequential structure

To verify sequential structure, we train on original data and test on both original and randomly permuted (within each context of 10) test sets. We also train on permuted data and test on original. Results are averaged over multiple random initializations.

Method

Our proposed approach is to meta-train a generator to produce synthetic datasets on which the target model exhibits the best generalization performance, i.e., datasets to which it is inductively biased.

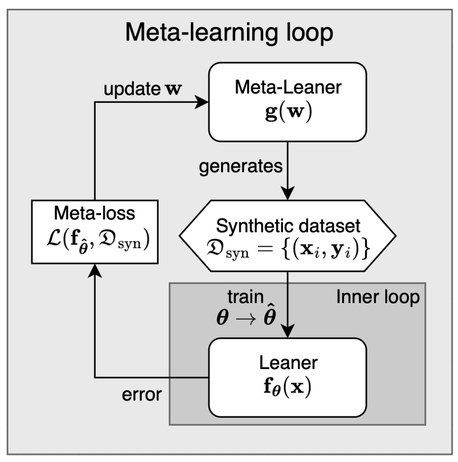

Figure: Meta-learning scheme.

Algorithm 1 Meta-learning for inductive bias extraction

Initialize generator $\mathbf{g}(\mathbf{w})$ with random parameters $\mathbf{w}$
while step $<$ total steps **do**
  Generate synthetic dataset $\mathfrak{D}_{\text{syn}} = \{(\mathbf{x}_i, \mathbf{y}_i)\}$ with C classes and N samples per class
  Split $\mathfrak{D}_{\text{syn}}$ into training set $\mathfrak{D}_{\text{train}}$ and test set $\mathfrak{D}_{\text{test}}$
  Re-initialize target model $\mathbf{f}_\theta$ from scratch
  Train $\mathbf{f}_\theta$ on $\mathfrak{D}_{\text{train}}$ in the inner loop $\to$ trained model $\mathbf{f}_{\hat{\theta}}$
  Compute meta-loss: $\mathcal{L}_{\text{meta}} = \mathcal{L}_g(\mathbf{f}_{\hat{\theta}}, \mathfrak{D}_{\text{test}}) + \lambda \mathcal{R}(\mathfrak{D}_{\text{syn}})$
  Take $\mathbf{w}$ gradient step on meta-loss
end while

CNNs

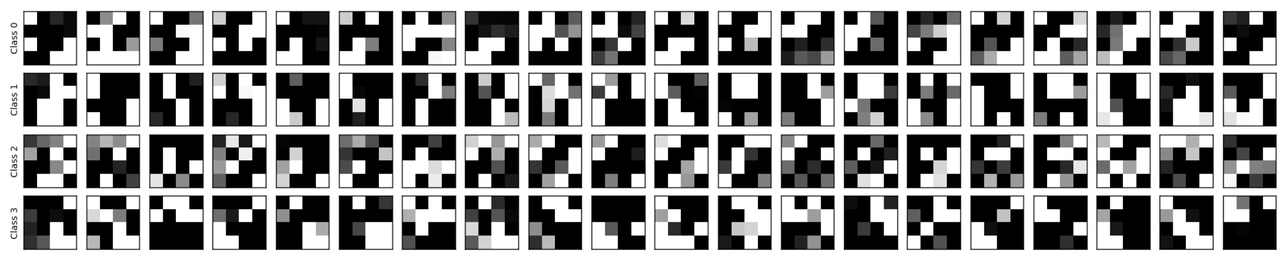

Figure: Obtained 4x4 dataset for cross-entropy and average intra-class distance regularization.

CNN Architecture: Conv2d with kernel 2x2, with stride 2x2.

Setup	Original	Shuffled
CE+Dist	$\mathbf{98.87 \pm 5.33}$	$\mathbf{98.87 \pm 5.33}$
MSE+Dist	$\mathbf{99.03 \pm 5.22}$	$\mathbf{99.03 \pm 5.22}$
CE+SVD	$\mathbf{99.33 \pm 4.62}$	$\mathbf{99.33 \pm 4.62}$
MSE+SVD	$\mathbf{98.96 \pm 5.46}$	$\mathbf{98.96 \pm 5.46}$
MLP	$\mathbf{62.24 \pm 14.73}$	$\mathbf{78.76 \pm 13.02}$

Table: Accuracy comparison on original and randomly shuffled data for all meta-loss configurations.

Locality

To verify locality, for each meta-loss configuration, we train the model on the original training set and evaluate on both original and randomly shuffled ($\mathbf{2 \times 2}$ block permutation) test sets. Results are averaged over 1000 random train-test splits.

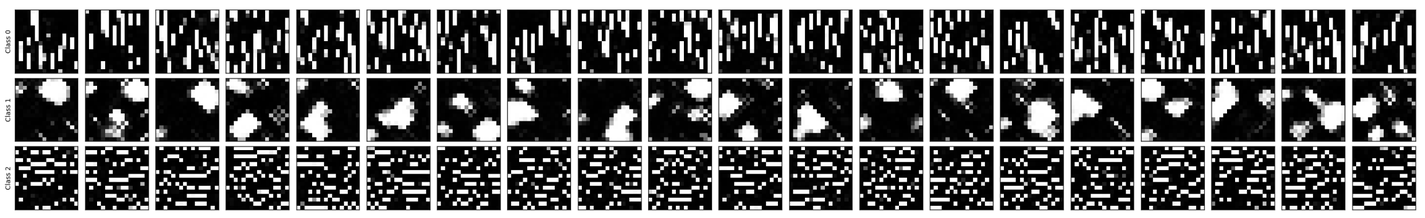

Figure: Obtained 16x16 dataset for cross-entropy and average intra-class distance regularization.

CNN Architecture: Two Conv2d layers (32/64 channels, 3x3 kernel, padding=1), adaptive max pooling, and a 1x1 Conv2d classifier.

Model	Centered	Random shift ($\pm$4px)
CNN	$\mathbf{99.65 \pm 4.07}$	$\mathbf{99.62 \pm 4.35}$
MLP	$\mathbf{14.72 \pm 7.33}$	$\mathbf{22.02 \pm 10.13}$

Table: Accuracy comparison on original and randomly shifted data.

Translation invariance

To verify translation invariance, we first embed 16x16 images into the center of 24x24 canvases and train the model on these centered images. We then evaluate on both centered and randomly shifted (within the canvas) test sets.

novokshonova.ak@phystech.edu

github.com/intsystems/2026-Project-205